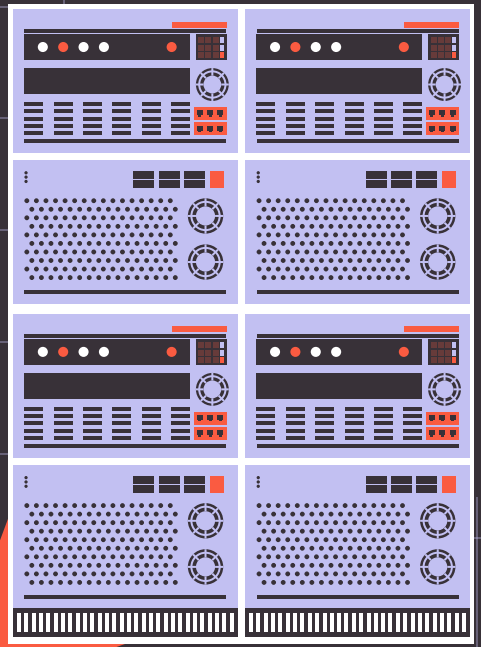
COURSE

FLASK WEB APPS WITH PYTHON



From your first route to a free live deployment

University of Liechtenstein

Dr. Pejman Ebrahimi

COURSE ROADMAP

01 BASICS
How the web works and what Flask is

02 SETUP
VS Code, Git Bash and flask_env

03 FIRST APP
Hello Flask, routes and HTTP methods

04 TEMPLATES
Jinja2, static files, forms, sessions

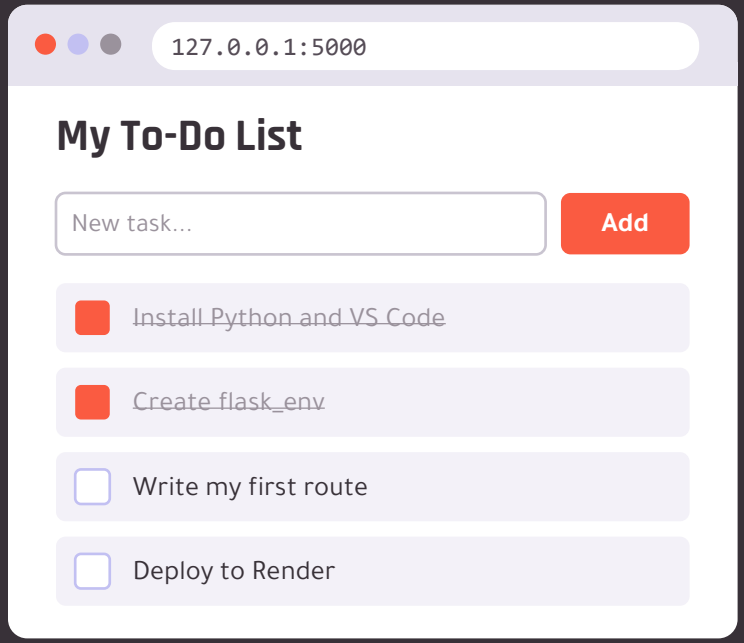
05 DATABASE
SQLite and the To-Do mini project

06 DEPLOY
Git, GitHub and free hosting

WHAT YOU WILL BUILD

A working **To-Do web app**, built step by step and deployed online for free.

- Set up **VS Code + Git Bash + flask_env**
- Write routes, views and Jinja2 templates
- Handle forms, flash messages and sessions
- Store data in SQLite with Flask-SQLAlchemy
- Expose a small JSON API
- Push to GitHub and deploy on **Render, Vercel or PythonAnywhere**



01

THE BASICS

How the web works and what Flask is



University of Liechtenstein

Dr. Pejman Ebrahimi

HOW THE WEB WORKS



BROWSER

the client: Chrome, Edge, Firefox

HTTP REQUEST `GET /hello`

HTTP RESPONSE `200 OK + HTML`



FLASK APP

the server: your Python code

URL

the address: https://site.com/hello?name=Ana

METHOD

what to do: GET reads, POST sends data

STATUS

the result: 200 OK, 302 redirect, 404, 500

WHAT IS FLASK?

Flask is a lightweight **micro web framework** for Python, maintained by the Pallets project.

- "Micro" means a small, simple core: you add only what you need
- A full web app can start in one Python file
- Huge ecosystem of extensions (database, login, forms, APIs)
- Great first framework: the concepts transfer to Django and FastAPI

WERKZEUG

Request, response, routing and the dev server (WSGI toolkit)

JINJA2

Template engine: turns HTML + variables into pages

CLICK

Command line: powers flask run and custom commands

EXTENSIONS

Flask-SQLAlchemy, Flask-Login, Flask-WTF and more

FLASK VS DJANGO VS FASTAPI

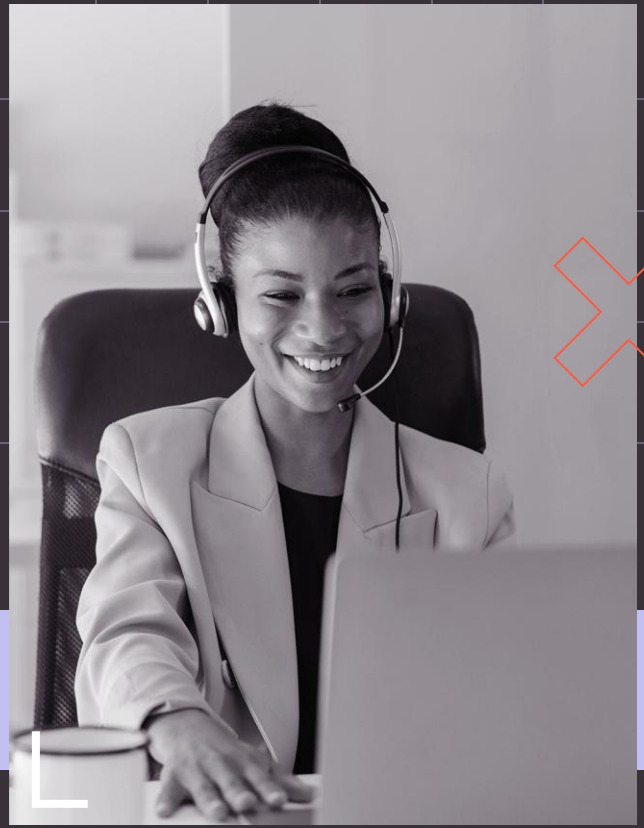
	FLASK	DJANGO	FASTAPI
Philosophy	Micro, flexible, you choose	Batteries included	API-first, type hints
Learning curve	Gentle	Steeper	Moderate
Database / admin	Via extensions	Built-in ORM + admin	Via libraries
Templates (HTML)	Jinja2 built in	Django templates	Optional (Jinja2)
Best for	Small to mid apps, prototypes, APIs	Large sites with auth and admin	Fast JSON APIs, async

Rule of thumb: learn Flask first to understand how web apps really work.

02

SETUP

VS Code, Git Bash and flask_env



WHAT YOU NEED



PYTHON 3.10+

python.org/downloads

Windows: tick “Add python.exe to PATH”



VS CODE

code.visualstudio.com

Install the “Python” extension (Microsoft)



GIT + GIT BASH

git-scm.com

Windows installer includes Git Bash



GITHUB ACCOUNT

github.com (free)

Needed later for deployment

macOS / Linux: use the built-in Terminal instead of Git Bash. Commands are the same, except where a slide shows a separate macOS line.

CHECK YOUR TOOLS IN GIT BASH

Git Bash

```
$ python --version    # Python 3.12.x
$ pip --version
$ git --version
$ code --version      # VS Code CLI
```

TROUBLESHOOTING

`python` opens the Microsoft Store? Turn off App execution aliases in Windows Settings, or use `py`.

macOS / Linux: type `python3` and `pip3`.

Every command in this course starts with **$** (your prompt). Type only what comes after it. Text after # is a comment.

CREATE THE PROJECT AND FLASK_ENV

Git Bash

```
$ mkdir flask_todo
$ cd flask_todo
$ python -m venv flask_env

# activate it (Windows, Git Bash)
$ source flask_env/Scripts/activate
# activate it (macOS / Linux)
$ source flask_env/bin/activate

(flask_env) $   # you are inside the env
(flask_env) $ deactivate   # leave it
```

WHY A VIRTUAL ENV?

- Each project gets its own private set of packages
- No version conflicts between projects
- Easy to rebuild anywhere from requirements.txt
- Activate `flask_env` every time you open a new terminal

INSTALL FLASK

Git Bash

```
(flask_env) $ python -m pip install --upgrade pip
(flask_env) $ pip install flask
(flask_env) $ flask --version
(flask_env) $ pip freeze > requirements.txt
```

.gitignore

```
flask_env/
__pycache__/
*.db
instance/
.env
```

REQUIREMENTS.TXT

Lists exact package versions. Anyone (or any server) can recreate the env with `pip install -r requirements.txt`

.GITIGNORE

Keeps flask_env, caches, local databases and secrets out of Git. Create it in the project root.

CONFIGURE VS CODE

1

OPEN THE FOLDER

File > Open Folder, or type `code .` in Git Bash

2

GIT BASH AS TERMINAL

Ctrl+Shift+P > **Terminal: Select Default Profile** > Git Bash

3

PICK THE INTERPRETER

Ctrl+Shift+P > **Python: Select Interpreter** > flask_env

4

OPEN A NEW TERMINAL

Ctrl+` (backtick). The prompt should show `(flask_env)`

USEFUL EXTENSIONS

- **Python** (Microsoft): run, debug, IntelliSense
- **Pylance**: smart autocompletion
- **Jinja** or **Better Jinja**: template highlighting
- **SQLite Viewer**: open .db files

PROJECT STRUCTURE

Explorer

```
flask_todo/
├── flask_env/          # virtual env
├── app.py              # entry point
├── requirements.txt
├── .gitignore
├── templates/          # HTML (Jinja2)
│   ├── base.html
│   └── index.html
├── static/             # CSS, JS, images
│   └── style.css
└── instance/           # SQLite db
```

TEMPLATES/

Flask looks here for HTML files used by render_template().

STATIC/

Files sent as they are: CSS, JavaScript, images.

INSTANCE/

Created by Flask for local data such as the SQLite database.

03

FIRST APP

Hello Flask, routes and
HTTP methods



University of Liechtenstein

Dr. Pejman Ebrahimi

HELLO, FLASK!

app.py

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Hello, Flask!</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

1. **Import:** the Flask class
2. **Create the app:** `__name__` tells Flask where to find files
3. **Route:** `@app.route("/")` maps a URL to a function
4. **View function:** returns what the browser shows
5. **Run:** only when the file is started directly

RUN YOUR APP

Git Bash

```
(flask_env) $ flask --app app run --debug
 * Running on http://127.0.0.1:5000
 * Debugger is active!

# alternative: run the file directly
(flask_env) $ python app.py
```

OPEN IT

Ctrl+click `http://127.0.0.1:5000` in the terminal or paste it in your browser.

Stop the server with **Ctrl+C**.

DEBUG MODE

Auto-reload on save + interactive error page. **Never enable debug in production.** Port 5000 busy? Add `--port 5001`.

ROUTES AND DYNAMIC URLS

app.py

```python
@app.route("/about")
def about():
    return "About page"

@app.route("/user/<name>")
def greet(name):
    return f"Hello, {name}!"

@app.route("/post/<int:post_id>")
def show_post(post_id):
    return f"Post number {post_id}"
```

CONVERTER	ACCEPTS
`string`	text without / (default)
`int`	positive integers
`float`	decimal numbers
`path`	text including /
`uuid`	UUID strings

Try: /user/Ana and /post/42. What happens with /post/abc?

HTTP METHODS: GET VS POST

GET

Reads data

Values in the URL: ?q=flask

Can be bookmarked

POST

Sends data

Values in the request body

Used by forms that change data

app.py

```python
from flask import request

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = request.form["username"]
        return f"Welcome, {user}!"
    return "Show the login form here"

# shortcuts (Flask 2.0+)
@app.get("/items")    # = methods=["GET"]
@app.post("/items")   # = methods=["POST"]
```

REQUEST, REDIRECT AND JSON

app.py

```python
from flask import request, redirect, url_for, jsonify

@app.route("/search")
def search():
    q = request.args.get("q", "")  # /search?q=flask
    return f"You searched for: {q}"

@app.route("/old-home")
def old_home():
    return redirect(url_for("home"))

@app.route("/api/status")
def status():
    return jsonify(ok=True, version="1.0")
```

`request.args`
URL query values

`request.form`
submitted form fields

`request.get_json()`
JSON body of an API call

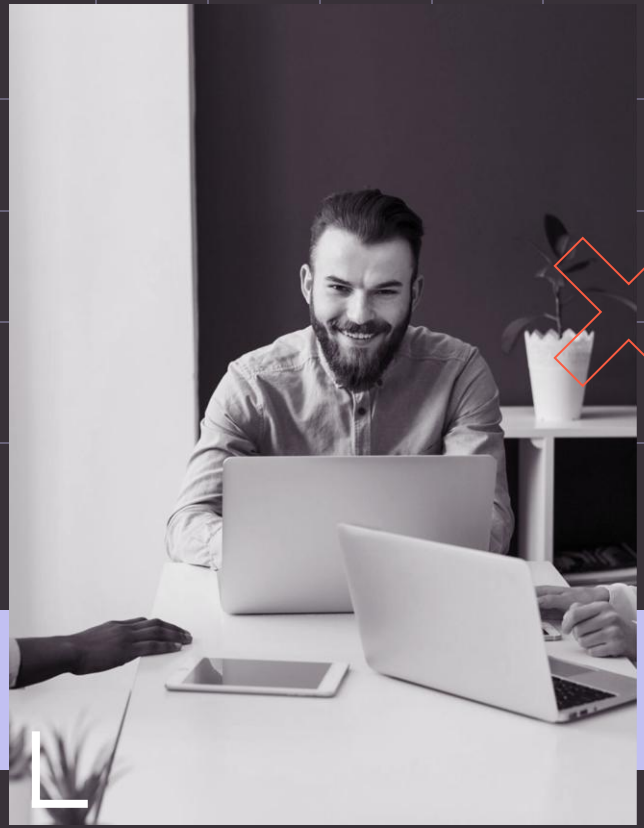
`url_for("home")`
builds a URL from a function name

`redirect(...)`
sends the browser to another URL

`jsonify(...)`
returns a JSON response

04 TEMPLATES & FORMS

Jinja2, static files, user input and sessions



University of Liechtenstein

Dr. Pejman Ebrahimi

JINJA2 TEMPLATES

app.py

```python
from flask import render_template

@app.route("/hello/<name>")
def hello(name):
    fruits = ["apple", "kiwi", "mango"]
    return render_template("hello.html",
        name=name, fruits=fruits)
```

templates/hello.html

```jinja
<h1>Hello {{ name|title }}!</h1>
{% if fruits %}
<ul>
  {% for f in fruits %}
  <li>{{ loop.index }}. {{ f }}</li>
  {% endfor %}
</ul>
{% else %}
<p>No fruit today.</p>
{% endif %}
```

`{{ ... }}`
print a value

`{% ... %}`
logic: if, for, block

`{# ... #}`
comment

`| filter`
upper, title, length

TEMPLATE INHERITANCE

templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>{% block title %}My App{% endblock %}</title>
  <link rel="stylesheet"
    href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <nav><a href="{{ url_for('home') }}">Home</a></nav>
  <main>{% block content %}{% endblock %}</main>
</body>
</html>
```

templates/index.html

```
{% extends "base.html" %}

{% block content %}
  <h1>Welcome!</h1>
{% endblock %}
```

Write the layout **once**. Child pages only fill the `blocks`. Change the menu in base.html and every page updates.

STATIC FILES: CSS, JS, IMAGES

templates/base.html

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
<script src="{{ url_for('static', filename='js/app.js') }}"></script>
<img src="{{ url_for('static', filename='img/logo.png') }}" alt="Logo">
```

Explorer

```
static/
├── css/
│   └── style.css
├── js/
│   └── app.js
└── img/
    └── logo.png
```

URL_FOR

Builds the right /static/ path, also online.

STYLE FAST

Link Bootstrap or Pico.css from a CDN.

NO CHANGE?

Browser cache: hard refresh with Ctrl+F5.

HANDLING FORMS

templates/contact.html

```
<form method="post">
  <input name="email" type="email">
  <textarea name="message"></textarea>
  <button type="submit">Send</button>
</form>
```

app.py

```
@app.route("/contact", methods=["GET", "POST"])
def contact():
    if request.method == "POST":
        email = request.form.get("email", "")
        msg = request.form.get("message", "")
        if not msg.strip():
            flash("Message is empty", "error")
            return redirect(url_for("contact"))
        flash("Thanks, message sent!", "ok")
        return redirect(url_for("home"))
    return render_template("contact.html")
```

POST / REDIRECT / GET

After a successful POST, `redirect`. Refreshing the page will not submit the form twice. Always validate on the server.

FLASH MESSAGES AND SESSIONS

app.py

```python
import os
from flask import flash, session

app.secret_key = os.environ.get(
    "SECRET_KEY", "dev-change-me")

@app.route("/visits")
def visits():
    session["n"] = session.get("n", 0) + 1
    return f"Visits: {session['n']}"
```

templates/base.html

```jinja
{% with msgs = get_flashed_messages(
     with_categories=true) %}
  {% for cat, m in msgs %}
    <div class="alert {{ cat }}">
      {{ m }}
    </div>
  {% endfor %}
{% endwith %}
```

SECRET_KEY

Signs the session cookie. Keep it secret.

FLASH

One-time message shown on the next page.

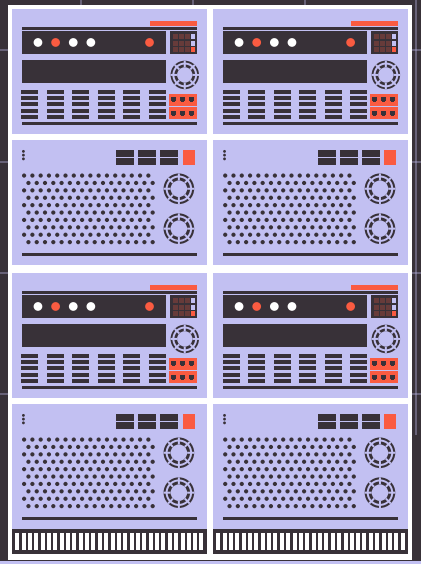
SESSION

Small per-user data stored in a cookie.

05

DATABASE & PROJECT

SQLite and the To-Do project



University of Liechtenstein

Dr. Pejman Ebrahimi

DATABASES WITH FLASK-SQLALCHEMY

Git Bash

```
(flask_env) $ pip install flask-sqlalchemy
```

app.py

```python
from flask_sqlalchemy import SQLAlchemy

app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///todo.db"
db = SQLAlchemy(app)

class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    done = db.Column(db.Boolean, default=False)

with app.app_context():
    db.create_all()   # creates instance/todo.db
```

PYTHON CLASS

```
Task(title="Buy milk")
```



TABLE: task

id	title	done
1	Buy milk	0
2	Learn Flask	1

ORM: write Python, not SQL

CRUD: CREATE, READ, UPDATE, DELETE

ACTION	CODE
Create	`db.session.add(Task(title="Buy milk")); db.session.commit()`
Read all	`tasks = db.session.scalars(db.select(Task)).all()`
Read one	`task = db.get_or_404(Task, task_id)`
Filter	`db.session.scalars(db.select(Task).filter_by(done=False)).all()`
Update	`task.done = True; db.session.commit()`
Delete	`db.session.delete(task); db.session.commit()`

Remember: nothing is saved until you call `db.session.commit()`

MINI PROJECT: TO-DO APP

ROUTE	METHOD	DOES
/	GET	List all tasks
/add	POST	Create a task
/toggle/<id>	POST	Mark done / undone
/delete/<id>	POST	Delete a task
/api/tasks	GET	Tasks as JSON

BUILD IT IN 5 STEPS

1. Create flask_todo + flask_env
2. pip install flask flask-sqlalchemy
3. Write app.py (next 2 slides)
4. Add base.html, index.html, style.css
5. Run, test, freeze requirements

PROJECT: APP.PY (1/2)

app.py (setup + model)

```python
import os
from flask import Flask, render_template, request, redirect, url_for, flash, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "dev-change-me")
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///todo.db"
db = SQLAlchemy(app)

class Task(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    done = db.Column(db.Boolean, default=False)

with app.app_context():
    db.create_all()
```

PROJECT: APP.PY (2/2)

app.py (routes)

```python
@app.route("/")
def index():
    tasks = db.session.scalars(
        db.select(Task)).all()
    return render_template(
        "index.html", tasks=tasks)

@app.post("/add")
def add():
    title = request.form["title"].strip()
    if title:
        db.session.add(Task(title=title))
        db.session.commit()
        flash("Task added!")
    return redirect(url_for("index"))
```

app.py (routes)

```python
@app.post("/toggle/<int:task_id>")
def toggle(task_id):
    task = db.get_or_404(Task, task_id)
    task.done = not task.done
    db.session.commit()
    return redirect(url_for("index"))

@app.post("/delete/<int:task_id>")
def delete(task_id):
    task = db.get_or_404(Task, task_id)
    db.session.delete(task)
    db.session.commit()
    return redirect(url_for("index"))

if __name__ == "__main__":
    app.run(debug=True)
```

PROJECT: BASE.HTML AND STYLE.CSS

templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>To-Do</title>
  <link rel="stylesheet" href=
  "{{ url_for('static',
     filename='style.css') }}">
</head>
<body>
  {% for m in get_flashed_messages() %}
    <p class="flash">{{ m }}</p>
  {% endfor %}
  {% block content %}{% endblock %}
</body>
</html>
```

static/style.css

```
body {
  font-family: system-ui, sans-serif;
  max-width: 640px;
  margin: 40px auto;
}
h1 { color: #FA5B41; }
form { display: inline; }
li { list-style: none; padding: 6px 0; }
.done { text-decoration: line-through; }
.flash { background: #C2C0F2;
         padding: 8px; }
```

PROJECT: INDEX.HTML

templates/index.html

```
{% extends "base.html" %}
{% block content %}
<h1>My To-Do List</h1>
<form method="post" action="{{ url_for('add') }}">
  <input name="title" placeholder="New task" required> <button>Add</button>
</form>
<ul>
  {% for t in tasks %}
  <li class="{{ 'done' if t.done }}">{{ t.title }}
    <form method="post" action="/toggle/{{ t.id }}"><button>Done</button></form>
    <form method="post" action="/delete/{{ t.id }}"><button>Delete</button></form>
  </li>
  {% else %}<li>No tasks yet</li>{% endfor %}
</ul>
{% endblock %}
```

PROJECT: RUN AND TEST

Git Bash

```
(flask_env) $ pip install flask flask-sqlalchemy
(flask_env) $ flask --app app run --debug
(flask_env) $ pip freeze > requirements.txt
```

OPEN

http://127.0.0.1:5000

Your To-Do app is running locally.

ADD

Type a task and click Add: it appears with a flash message

DONE

Click Done: the title is crossed out

DELETE

Click Delete: the task disappears

RESTART

Stop and start the server: tasks are still there (SQLite)

BONUS: A JSON API

app.py

```python
@app.get("/api/tasks")
def api_list():
    tasks = db.session.scalars(
        db.select(Task)).all()
    return jsonify([
        {"id": t.id, "title": t.title,
         "done": t.done} for t in tasks])

@app.post("/api/tasks")
def api_add():
    data = request.get_json()
    task = Task(title=data["title"])
    db.session.add(task)
    db.session.commit()
    return jsonify(id=task.id), 201
```

Git Bash (2nd terminal)

```
$ curl localhost:5000/api/tasks

$ curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"title": "Learn APIs"}' \
  localhost:5000/api/tasks
```



WHY AN API?

Mobile apps, JavaScript front ends and other programs can use your data.

ERRORS AND DEBUGGING

app.py

```python
from flask import abort

@app.errorhandler(404)
def not_found(e):
    return render_template(
        "404.html"), 404
```

Read the **last line** of the traceback first: it names the error. Then look up the file and line number above it.

ERROR	USUAL FIX
No module named 'flask'	Activate flask_env / select interpreter
TemplateNotFound	Put the file inside templates/
Address already in use	Stop the other server or --port 5001
405 Method Not Allowed	Add methods=["POST"] to the route
UndefinedError (Jinja)	Pass the variable in render_template
session needs secret key	Set app.secret_key

BLUEPRINTS AND EXTENSIONS

blueprints

```python
# routes/auth.py
from flask import Blueprint

auth = Blueprint("auth", __name__,
                 url_prefix="/auth")

@auth.route("/login")
def login():
    return "Login page"

# app.py
from routes.auth import auth
app.register_blueprint(auth)  # /auth/login
```

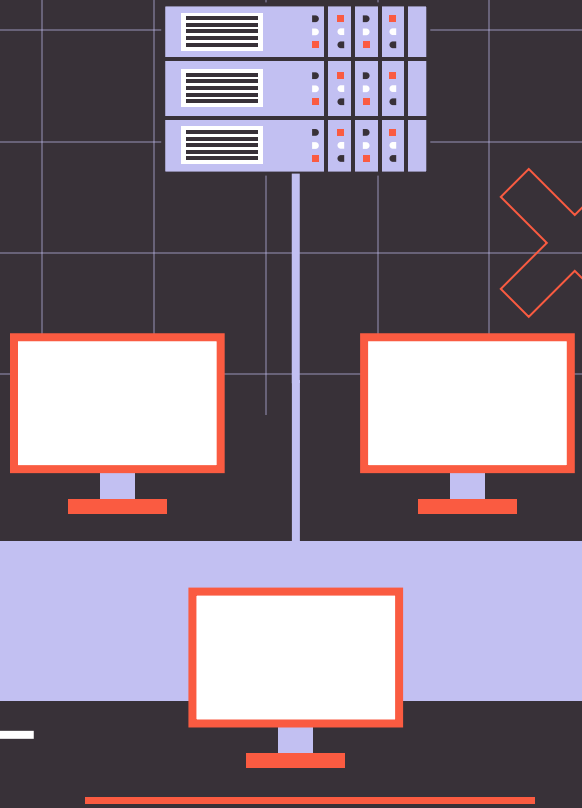
POPULAR EXTENSIONS

- **Flask-Login** user sessions and login
- **Flask-WTF** forms with CSRF protection
- **Flask-Migrate** change DB tables safely
- **python-dotenv** load settings from .env
- **Flask-CORS** allow API calls from other sites

06

GIT & DEPLOYMENT

Put your app online for free



University of Liechtenstein

Dr. Pejman Ebrahimi

PREPARE FOR PRODUCTION

DEBUG OFF

Never run debug=True on a server

SECRET FROM ENV

Read SECRET_KEY from an environment variable

GUNICORN

A real WSGI server for Linux hosts. Start command: gunicorn app:app

requirements.txt

pip freeze after every new package

PYTHON VERSION

Use the same version locally and online

.gitignore

flask_env and *.db stay out of Git

Git Bash

```
(flask_env) $ pip install gunicorn && pip freeze > requirements.txt
```

SAVE YOUR CODE ON GITHUB

Git Bash

```
# one time only
$ git config --global user.name "Your Name"
$ git config --global user.email "you@uni.li"

$ git init
$ git add .
$ git commit -m "First version of my To-Do app"
$ git branch -M main
$ git remote add origin https://github.com/YOU/flask_todo.git
$ git push -u origin main

# later: add, commit, push again
```

FIRST

On github.com click New repository. Name it flask_todo and leave it empty.

CHECK

Run `git status` before committing: flask_env must not be listed.

FREE DEPLOYMENT OPTIONS

	RENDER	VERCEL	PYTHONANYWHERE
How it runs	Web service (container)	Serverless functions	Classic Python hosting
Free plan	Sleeps after 15 min idle, ~1 min wake-up	Hobby plan (personal use)	Beginner account, yourname.pythonanywhere.com
SQLite data	Lost on redeploy or restart	Not persistent	Persists
Deploy from	GitHub repo, auto-deploy on push	GitHub repo or vercel CLI	Git clone in a Bash console
Best for	Most Flask apps	APIs and sites without a local DB	Beginners, SQLite projects

Also free: Hugging Face Spaces (Docker). Free plans change often: check each site's pricing page before class.

DEPLOY ON RENDER

1	**NEW WEB SERVICE**	render.com > sign in with GitHub > **New > Web Service** > pick your repo
2	**SETTINGS**	Language: Python 3 · Instance type: **Free**
3	**BUILD COMMAND**	`pip install -r requirements.txt`
4	**START COMMAND**	`gunicorn app:app` (file:variable)
5	**ENVIRONMENT**	Add `SECRET_KEY`, then Deploy

LIVE URL

`your-app.onrender.com`
Every git push redeploys.

KEEP IN MIND

Free service sleeps when idle. Use Render Postgres for data that must last.

DEPLOY ON VERCEL

Git Bash

```
$ npm i -g vercel    # needs Node.js
$ vercel login
$ vercel             # preview deploy
$ vercel --prod      # production
```

Or without CLI: vercel.com/new > import your GitHub repo > Deploy.

WHAT VERCEL EXPECTS

- `app.py` at the root with a variable named `app`
- `requirements.txt` with flask (no gunicorn needed)
- Static files go in `public/`, e.g. public/static/style.css
- No vercel.json needed (zero-config Flask)
- **Local files do not persist: use a hosted database**

DEPLOY ON PYTHONANYWHERE

PythonAnywhere Bash console

```
$ git clone https://github.com/YOU/flask_todo.git
$ cd flask_todo
$ python3 -m venv flask_env
$ source flask_env/bin/activate
$ pip install -r requirements.txt
```

WSGI configuration file

```
import sys
sys.path.insert(0, "/home/YOU/flask_todo")
from app import app as application
```

1. **SIGN UP**
   Free Beginner account
2. **CONSOLES > BASH**
   run the commands on the left
3. **WEB > ADD WEB APP**
   Manual configuration, same Python version
4. **SET PATHS**
   Source: /home/YOU/flask_todo Virtualenv: .../flask_env
5. **EDIT WSGI FILE**
   then Reload: YOU.pythonanywhere.com

DEPLOYMENT TROUBLESHOOTING

READ THE LOGS

Every host has a Logs tab. The error is almost always there.

MISSING MODULE

ModuleNotFoundError: add the package to requirements.txt, commit, push.

START COMMAND

gunicorn app:app means file app.py, variable app.

SLOW FIRST LOAD

Free Render service is waking up. Wait about a minute.

CSS 404 ON VERCEL

Move static files into public/.

DATA DISAPPEARED

Ephemeral disk. Use a hosted database or PythonAnywhere.

COMMAND CHEAT SHEET

TASK	COMMAND (GIT BASH)
Create env	`python -m venv flask_env`
Activate (Windows)	`source flask_env/Scripts/activate`
Activate (macOS / Linux)	`source flask_env/bin/activate`
Install packages	`pip install flask flask-sqlalchemy gunicorn`
Save dependencies	`pip freeze > requirements.txt`
Run in debug mode	`flask --app app run --debug`
Push changes	`git add . && git commit -m "msg" && git push`
Production server	`gunicorn app:app`

PRACTICE AND NEXT STEPS

LEVEL 1

PERSONAL SITE

Home, About and Contact pages that share one base.html and a CSS file.

LEVEL 2

BETTER TO-DO

Add edit, due date and priority. Show open tasks first.

LEVEL 3

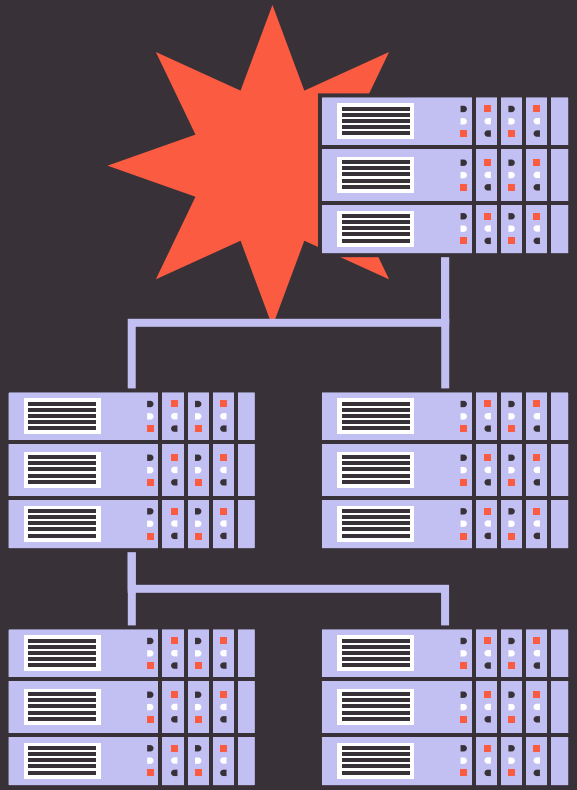
SHIP IT

Add the JSON API, deploy on Render or PythonAnywhere and share the link.

KEEP LEARNING

flask.palletsprojects.com official docs + tutorial **flask-sqlalchemy.readthedocs.io** database guide

jinja.palletsprojects.com template syntax **Flask Mega-Tutorial** by Miguel Grinberg (free online)



THANK YOU!

QUESTIONS? GET IN TOUCH

pejman.ebrahimi@uni.li
Dr. Pejman Ebrahimi
University of Liechtenstein